I. QUESTION (20 MINUTES)

```
1  public class Element {
2      int data;
3  }
4  public class Queue {
5      Element array[];
6      int first ;
7      int last ;
8      int N;
9  }
```

```
1  public class Node {
2      int data;
3      Node next;
4  }
5  public class Queue {
6      Node first ;
7      Node last;
8  }
```

Write the Java method

```
1  int minimum()
```

in Queue class where the method returns the minimum number in a queue. Write the function for both array and linked list implementations. Do not use any class or external methods except isEmpty().

II. QUESTION (25 MINUTES)

```
1  class Element {
2      int data;
3  }
4  class Queue {
5      Element* array;
6      int first ;
7      int last ;
8      int N;
9  }
```

```
1  class Node {
2      int data;
3      Node* next;
4  }
5  class Queue {
6      Node* first ;
7      Node* last;
8  }
```

Write the C++ methods

```
1  Element dequeue2nd()
2  Node* dequeue2nd()
```

in Queue class which removes and returns the second item from the queue. Write the function for both array and linked list implementations. Your methods should run in $\mathcal{O}(1)$ time. Do not use any class or external methods except isEmpty().

III. QUESTION (20 MINUTES)

```
1  class Tree {
2      TreeNode* root;
3  }
4  class TreeNode {
5      TreeNode* left;
6      TreeNode* right;
7      int data;
8  }
```

Write the recursive C++ method

```
1  int higherThanX(int X)
```

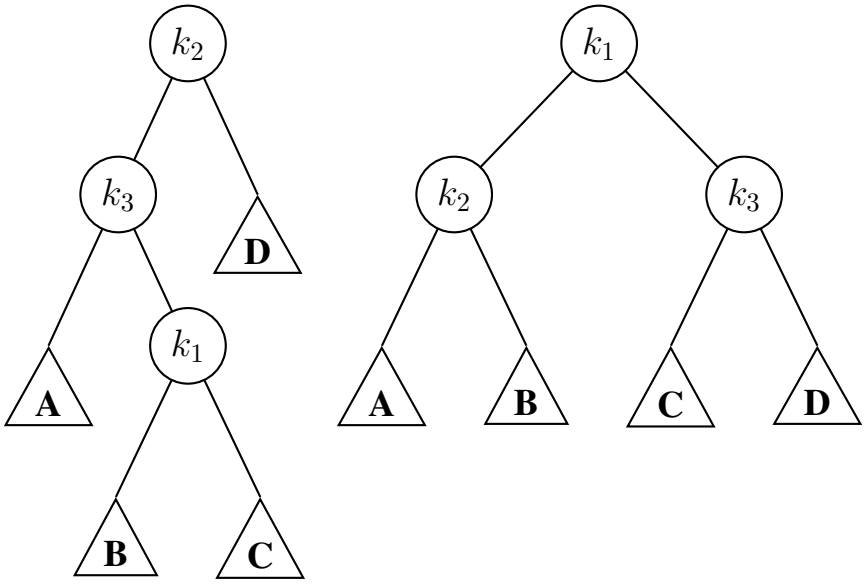
in TreeNode class, which returns the number of nodes in the binary search tree which have value larger than X. Your method should run in $\mathcal{O}(\log N + K)$ time, where N is total number of nodes and K is the number of nodes which have value larger than X in the tree. Do not use any class or external methods.

IV. QUESTION (20 MINUTES)

```
1  public class AvlTree {
2      AvlNode root;
3  }
4  public class AvlNode {
5      AvlNode left;
6      AvlNode right;
7      int data;
8      int height;
9  }
```

Write the Java method in AvlTree class for the following rotation in AVL trees. Your function should return node k_1. Update also the heights of subtrees k_1, k_2, and k_3. Do not use any class or external methods.

```
1  AvlNode rotate(AvlNode k2)
```



V. QUESTION (25 MINUTES)

```
1  public class Element {
2      int data;
3  }
4  public class Hash {
5      Element[] table;
6      boolean[] deleted;
7      int N;
8      int hashFunction(int value);
9  }
```

```
1  public class Node {
2      int data;
3      Node next;
4  }
5  public class LinkedList {
6      Node head;
7      Node tail;
8  }
9  public class Hash {
10     LinkedList[] table;
11     int N;
12     int hashFunction(int value);
13 }
```

Write the Java method

```
1  void deleteAll(int X)
```

in Hash class that deletes all elements having value *X*. Assume also that *X* can exist more than once in the hash table. Write the function for both array and linked list implementations. For array implementation assume that linear probing is used as the collision strategy. Do not use any class or external methods except hashFunction.

VI. QUESTION (20 MINUTES)

```
1  class Element {
2      int data;
3  }
4  class Hash {
5      Element **table;
6      bool* deleted;
7      int N;
8      int hashFunction(int value);
9  }
```

Write the C++ hash function

```
1  int hashFunctionItSelf()
```

in Hash class that maps the key values in an hash table into an hash value. Assume that the hash value of an hash table can be obtained first by summing up the key values of the elements in the hash table and then hashing the sum. Write the function for array implementation. Assume also that linear probing is used as the collision strategy. Do not use any class or external methods except hashFunction.